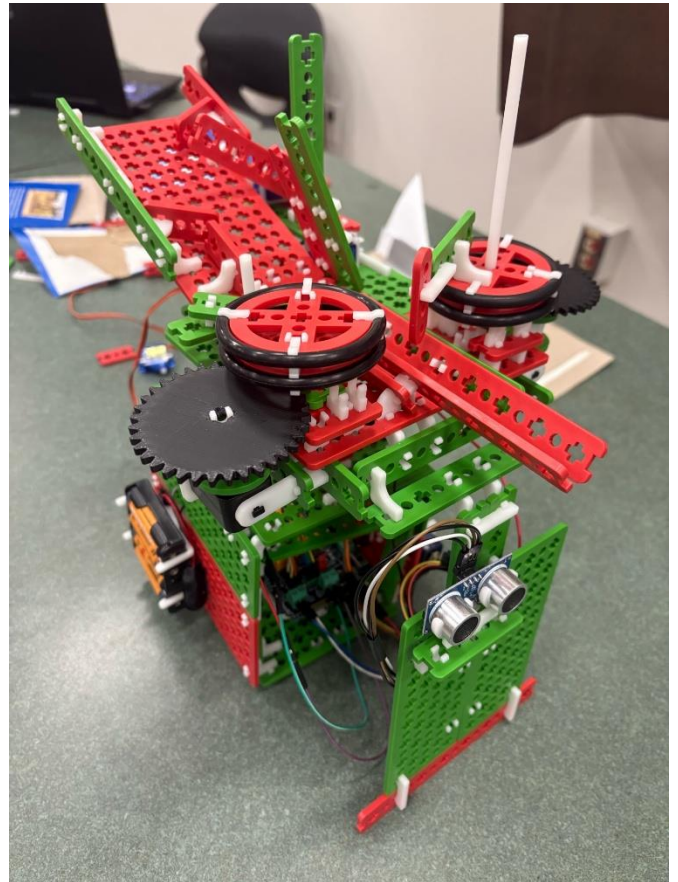
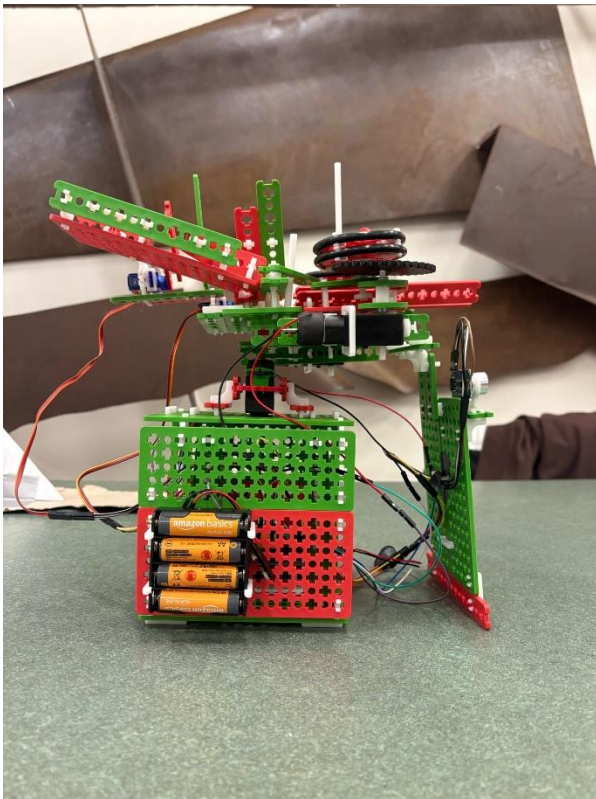


MEC101, Fall 2025 Robot Design Project Report

Team 14 - “The Projectiles”

Miles Carmody, Angelo Brescia, Axel Pesantez, Kenny Nguyen



A rotating turret that senses and shoots at targets when they are in front of the turret.

Our turret rotates with the use of a servo connected to the base of the turret. This servo rotates continuously unless an object is sensed in front of it. This is where our ultrasonic sensor comes in; it will sense an object within 35 cm of the sensor, and at that point the servo will stop rotating until the object is

moved out of the sensor's range. In addition to stopping the rotation of the sensor, it also triggers our reloading and firing mechanism. After an object is sensed and the servo stops, our reloading mechanism is first initiated. Our reload had to feed only one ball at a time into the firing chamber. We used two servos to make this possible. After the ball was loaded, our turret fired at our target. Following the firing of the turret and the target being knocked over, the turret continues to rotate. Our design's purpose is to have a turret that can hit a target with power and accuracy while also having the ability to continue its function even after it has hit the first target, allowing it to hit multiple targets.

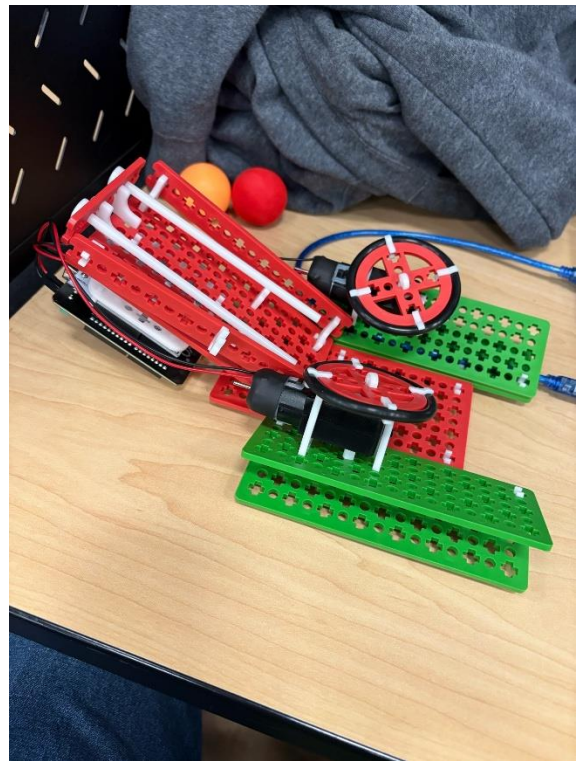
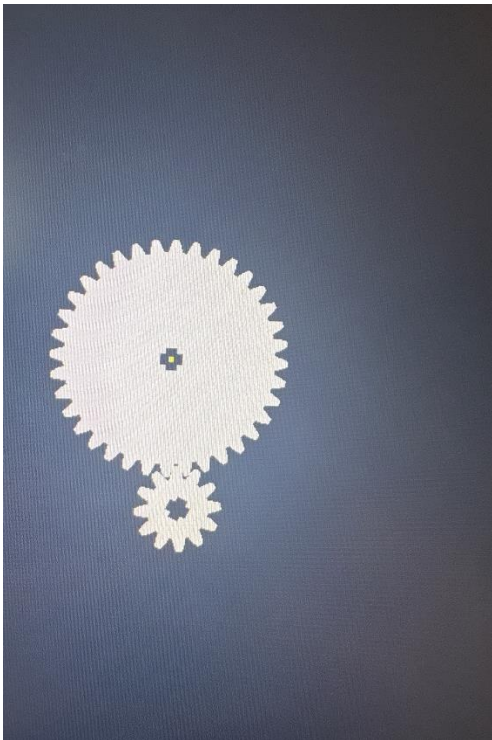


Fig 1. Inventor sketch of the 3d printed gears Fig 2. Our first design of the firing mechanism

Our early ideas for a firing mechanism were to feed a ball through two wheels connected to DC motors like a baseball launcher. We tested many ways to make this possible but fell short every time. Our original design had no gears and used the default speed of the DCS to spin the wheels. This speed was too

slow, so we incorporated gears in a 3:1 ratio to increase the speed. However, this also failed, as this speed was still too slow to provide meaningful results in our launch tests. Even though our original plan failed, we still used elements from it to create our final design. The final design for our firing mechanism kept the gears, wheels, and DC motors from the original, but we made an additional change. For this design we added a bar at the top of one of the wheels to add extra force on the ball as it is being fed through the two wheels. This increase in speed from the bar allowed the ball to have enough power to knock over our targets. In addition to speed and power, we needed a way to make the shots accurate. To achieve this, we added longer bars that extended past the wheels that made the path of the ball much more accurate to the center of the turret. This was crucial, as our sensor was also in the center of the turret.

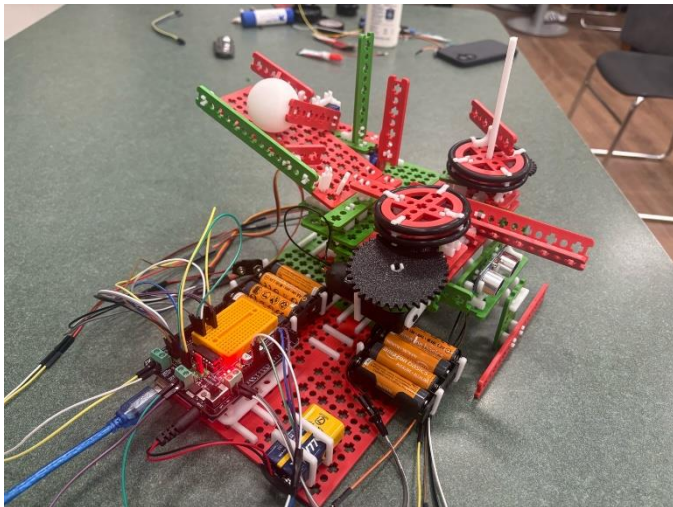


Fig3. Our Pre-Demo design didn't include the extended bar on the first servo

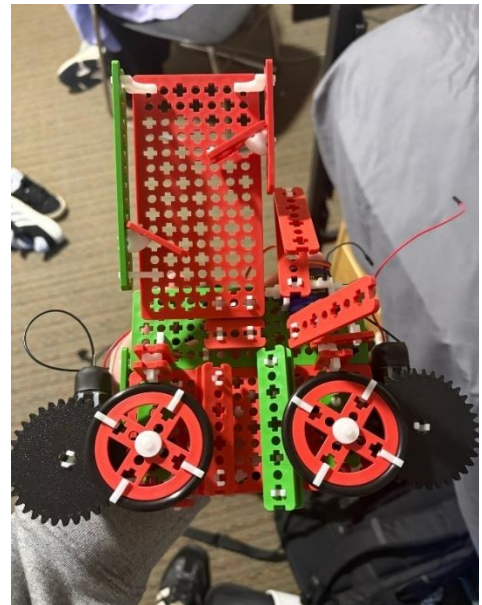


Fig4. Our original design which only included one servo

For our reloading mechanism, we started out only using one servo, as it would decrease the weight of the turret and make the coding simpler. This design choice changed as we had problems with feeding more than one ball at a

time. With one servo the recovery was not fast enough after the release of the first ball to stop the second. We knew at that point that we would have to add a second servo to the design. We added the second servo to a higher-up position when compared to the original but kept the same bar attachment that allowed for release and recovery. With the correct timing in the code, the servos lifted and dropped at the perfect time that allowed for the release of only one ball into the chamber. We also made the bar on the first servo thicker, as when it dropped, the bar would hold the ball in the chamber in place so that our firing mechanism would work correctly.

YouTube Link: <https://youtu.be/NEeeBO8jd7I?si=VI2Bcu5GnZW9wpAB>

Materials used:

Quantity	Source	Description
4	SnappyXO kit	Red wheel
2	SnappyXO kit	Square dc motor mount
2	SnappyXO kit	Bar dc motor mount
4	SnappyXO kit	6V battery mount
2	SnappyXO kit	9V battery mount
1	SnappyXO kit	Arduino mount
4	SnappyXO kit	4x8 plate
7	SnappyXO kit	3x8 plate
2	SnappyXO kit	8x bar
3	SnappyXO kit	7x bar
1	SnappyXO kit	6x bar
6	SnappyXO kit	5x bar
4	SnappyXO kit	4x bar
6	SnappyXO kit	3x bar
10	SnappyXO kit	2x bar
2	SnappyXO kit	Mini servo mount
1	SnappyXO kit	Large servo mount
1	SnappyXO kit	Ultra sonic sensor holder
26	SnappyXO kit	L clip
44	SnappyXO kit	H clip
22	SnappyXO kit	C clip
2	SnappyXO kit	Small angle clip
7	SnappyXO kit	Larger angle clip
2	SnappyXO kit	Medium length rod
1	SnappyXO kit	Long rod

6	SnappyXO kit	Loose spacer
2	SnappyXO kit	Tight spacer
2	3d printed	Large gear
2	3d printed	Pinion
2	Mechatronics kit	Dc motor
2	Mechatronics kit	Small servo
1	Mechatronics kit	Large servo
1	Mechatronics kit	Motor shield
1	Mechatronics kit	Arduino uno
2	Mechatronics kit	6V battery pack
1	Mechatronics kit	9V battery
35	Mechatronics kit	Wires (female to male)
3	Mechatronics kit	Battery connection wires

Code:

```
#include <Servo.h>

int pos = 0;
int state = 1;
Servo servo_2;
bool ready = false;

Servo servo_10; // FIRST RELEASE

Servo servo_9; // SECOND RELEASE

int motor1Pin = 4; // H-bridge leg 1
int motor2Pin = 5; // H-bridge leg 2
int motor3Pin = 13;
int motor4Pin = 12;
int speedPin = 6; // H-bridge enable pin
int speed2Pin = 11;

int trig = A3;
int echo = A2;
```

```

float cm;
float timer;

int on = 1;
int ledAuto = 7;
int counter = 0;
long microsecondsToCentimeters(float microseconds) {
    // The speed of sound is 34300 cm/s
    return (microseconds / 1000000.0 * 34300.0) / 2.0;
}

void setup() {
    pinMode(motor1Pin, OUTPUT);
    pinMode(motor2Pin, OUTPUT);
    pinMode(speedPin, OUTPUT);
    pinMode(motor3Pin, OUTPUT);
    pinMode(motor4Pin, OUTPUT);
    pinMode(speed2Pin, OUTPUT);
    /*digitalWrite(speedPin, HIGH);
    digitalWrite(speed2Pin, HIGH);*/
    // Sonic radar pin
    pinMode(trig, OUTPUT);
    pinMode(echo, INPUT);

    Serial.begin(9600);
    servo_2.attach(A0, 500, 2500);
    for (pos = 0; pos <= 90; pos += 1) {
        // tell servo to go to position in variable 'pos'
        servo_2.write(pos);
        // wait 15 ms for servo to reach the position
        delay(20); // Wait for 15 millisecond(s)
    }

    servo_10.attach(10, 500, 2500);
    servo_10.write(140);
    servo_9.attach(9, 500, 2500);
    servo_9.write(140);
}

void loop() {
    digitalWrite(trig, LOW);
    delayMicroseconds(2);

```

```

digitalWrite(trig, HIGH);
delayMicroseconds(10);
digitalWrite(trig, LOW);
timer = pulseIn(echo, HIGH, 50000);

// convert the time into a distance
cm = microsecondsToCentimeters(timer);
while ((cm > 35) || (cm == 0)) {
    digitalWrite(speedPin, LOW);
    digitalWrite(speed2Pin, LOW);
    servo_2.write(pos);
    if (state == 1) {
        pos++;
    } else {
        pos--;
    }
    if (pos == 180) {
        state = -1;
    }
    if (pos == 10) {
        state = 1;
    }

    digitalWrite(trig, LOW);
    delayMicroseconds(2);
    digitalWrite(trig, HIGH);
    delayMicroseconds(10);
    digitalWrite(trig, LOW);
    timer = pulseIn(echo, HIGH, 50000);

    // convert the time into a distance
    cm = microsecondsToCentimeters(timer);
    Serial.println("FINDING");
    delay(1);
}
while ((cm <= 35) && (cm != 0)) {
    digitalWrite(speedPin, LOW);
    digitalWrite(speed2Pin, LOW);

    //servo release
    servo_10.write(80);
    delay(750);
}

```

```

servo_10.write(130);
servo_9.write(80);

delay(700);
servo_9.write(120);
delay(450);
digitalWrite(speedPin, HIGH);
digitalWrite(speed2Pin, HIGH);
digitalWrite(motor1Pin, LOW);    // set leg 1 of the H-bridge low
digitalWrite(motor2Pin, HIGH);  // set leg 2 of the H-bridge high
digitalWrite(motor3Pin, LOW);
digitalWrite(motor4Pin, HIGH);
delay(750);

digitalWrite(trig, LOW);
delayMicroseconds(2);
digitalWrite(trig, HIGH);
delayMicroseconds(10);
digitalWrite(trig, LOW);
timer = pulseIn(echo, HIGH, 50000);

// convert the time into a distance
cm = microsecondsToCentimeters(timer);
/*if (cm > 50) {
    ready = false;
    break;
}*/
Serial.println("Shoot");
}
}

```

TinkerCAD: <https://www.tinkercad.com/things/8jA3A77suuU-mec-project-turret>

